COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	July 30, 2026	**DATE SUBMITTED:**	August 1, 2026
NAME:	Carlo Mari B. Asajar	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 4.1 STACKS

1. PROCEDURE OUTPUT

ILO A: Create a stack using the C++ STL

Definition from the official CPP documentation for the stack STL.

Stacks are a type of container adaptor, specifically designed to operate in a LIFO context (last-in first-out), where elements are inserted and extracted only from one end of the container.

Stacks are implemented as container adaptors, which are classes that use an encapsulated object of a specific container class as its underlying container, providing a specific set of member functions to access its elements. Elements are pushed/popped from the "back" of the specific container, which is known as the top of the stack.

The underlying container may be any of the standard container class templates or some other specifically designed container class. The container shall support the following operations:

empty
size
back
push_back
pop_back

The standard container classes vector, deque and list fulfill these requirements. By default, if no container class is specified for a particular stack class instantiation, the standard container deque is used. The member functions in the stack STL is shown in the figure below:

(constructor)	Constructor stack (public member function)
empty	Test whether container is empty (public member function)
size	Return size (public member function)

top	Access next element (public member function)
push	Insert element (public member function)
emplace	Construct and insert element (public member function)
pop	Remove top element (public member function)
swap	Swap the contents (public member function)

Member Functions

Test the stack STL operations through the given code below. Provide screenshot per operation, your observations and any other remarks in the output section of your activity template.

My Code:

Asajar_Seatwork4_1_ILOA.cpp > main()

```cpp
#include <iostream>
#include <stack> // call the stack from the STL

int main(){
    std::stack<int> newStack;
    std::stack<int> firstStack;
    std::stack<int> secondStack; // created an object

    // testing the empty() function to check if the stack is empty
    std::cout << "\nIs the stack empty? " << newStack.empty() << std::endl;

    // insert three elements into the stack
    newStack.push(3);
    std::cout << "Top of stack: " << newStack.top() << std::endl;
    newStack.push(2);
    std::cout << "Top of stack: " << newStack.top() << std::endl;
    newStack.push(1);
    std::cout << "Top of stack: " << newStack.top() << std::endl;

    // testing the empty() function again after inserting elements
    std::cout << "\nIs the stack empty? " << newStack.empty() << std::endl;
    newStack.push(4);
    newStack.push(5);
    newStack.push(6);

```

```
26      // size() -> returns the total number of elements in the stack
27      std::cout << "Size of the stack: " << newStack.size() << "." << std::endl
28
29      // swap -> swap the content of the public member function
30      firstStack.push(7);
31      secondStack.push(8);
32      firstStack.swap(secondStack);
33      std::cout << "\nSwapping..." << std::endl;
34      std::cout << "firstStack: " << firstStack.top() << std::endl;
35      std::cout << "secondStack: " << secondStack.top() << std::endl;
36
37      // emplace() -> constructs and inserts an element into the stack
38      std::cout << "\nEmplace of the stack..." << std::endl;
39      newStack.emplace(9);
40      std::cout << "Top of the stack: " << newStack.top() << std::endl;
41      std::cout << "Size of the stack: " << newStack.size() << std::endl << std
42
43      return 0;
44  }
```

My Output:

```
Is the stack empty? 1
Top of stack: 3
Top of stack: 2
Top of stack: 1

Is the stack empty? 0
Size of the stack: 6.

Swapping...
firstStack: 8
secondStack: 7

Emplace of the stack...
Top of the stack: 9
Size of the stack: 7
```

My Analysis:

The code shows the basic functions of the ***std::stack*** library. The code creates three stack objects which are ***newStack***, ***firstStack***, and ***secondStack***, which are used to test different stack operations. The ***empty()*** function is first used to check if the stack has no elements, and the output shows ***1*** because the stack is empty. The push() function is then used to add values ***3***, ***2***, and ***1*** into the stack, while the ***top()*** function displays the latest added value, showing the LIFO (Last In, First Out) behavior of stacks. After adding more elements like ***4***, ***5***, and ***6***, the ***size()*** function is used, and the output shows the total number of elements stored in the stack. The **swap()** function is also tested by exchanging the contents of ***firstStack*** and ***secondStack***, where the output shows that the values are successfully switched. Lastly, the ***emplace()*** function inserts the value ***9*** into the stack, and the ***top()*** and ***size()*** functions confirm the updated stack information.

ILO B: Develop C++ code that uses both arrays and linked lists to create a stack

In this section, we will have an implementation of stack through an array and a linked list.

B.1. Stacks using Arrays

Task:

- Modify the code given above to include a function that will display all elements in the stack.
- Provide a description of each operation provided.
- Include your output in the output section of your activity template.

My Code:

ILOB1 > ILOB1.cpp > main()

```cpp
#include <iostream>

const size_t maxCap = 100;
int stack[maxCap];
int top = -1, newData, elements;

// prototyping
bool isEmpty();
bool isFull();
void push();
void pop();
void Top();
void displayAll();

int main() {
    int choice;
    std::cout << "Enter the number of maximum elements for the new stack (max: 100): ";
    std::cin >> elements;

    while(true){
```

```
21          std::cout <<"\nStack Operations:"<<std::endl;
22          std::cout <<"1. Push \n2. Pop \n3. Top \n4. isEmpty \n5. isFull \n6. Display All" << std::endl;
23          std::cout << "Select option: ";
24          std::cin >> choice;
25
26          switch(choice){
27              case 1: push();
28              break;
29              case 2: pop();
30              break;
31              case 3: Top();
32              break;
33              case 4: std::cout << isEmpty() << std::endl;
34              break;
35              case 5: std::cout << isFull() << std::endl;
36              break;
37              case 6: displayAll();
38              break;
39              default: std::cout << "\nInvalid choice. Please try again." << std::endl;
40              break;
41          }
42      }
43
44      return 0;
45  }
46
47  bool isEmpty(){
48      // top is - 1
49      if(top == -1) {
50          std::cout <<"\nStack Underflow" << std::endl;
51          return true;
52      }
53      return false;
54  }
55
56  bool isFull(){
57      // not equal to the maxCap
```

```
58      if(top >= elements-1){
59          std::cout <<"\nStack Overflow";
60          return true;
61      }
62      return false;
63  }
64
65  void push(){
66
67      // check if the stack is full. if yes return an error
68      if (isFull()) return;
69
70      // ask the user for a value to be pushed into the stack
71      std::cout << "\nWhat number do you want to add to the stack? ";
72      std::cin >> newData;
73
74      // store inside the stack
75      stack[++top] = newData;
76  }
```

```
77
78  void pop(){
79
80      // check if there are no more elements inside the stack
81      if(isEmpty()) return;
82
83      //display the element to be popped:
84      std::cout << "\nPopping " <<stack[top] << "..." << std::endl;
85
86      // decrement top
87      top--;
88  }
89
90  void Top() {
91      // error checking
92      if(isEmpty()) return;
93
94      // display the top element
95      std::cout << "\nThe top element is " << stack[top] << "." << std::endl;
96  }
97
98  void displayAll() {
99      // display all elements
100     std::cout << "\nDisplaying Stacks..." << std::endl << std::endl;
101     for (int i = top; i >= 0; i--) {
102         std::cout << stack[i] << std::endl;
103     }
104 }
```

My Output:

Part 1:

```
Enter the number of maximum elements for the new stack (max: 100): 100

Stack Operations:
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display All
Select option: 4

Stack Underflow
1

Stack Operations:
1. Push
```

Part 2:

```
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display All
Select option: 1

What number do you want to add to the stack? 10
```

Part 3:

```
Stack Operations:
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display All
Select option: 1

What number do you want to add to the stack? 20

Stack Operations:
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display All
Select option: 1

What number do you want to add to the stack? 30

Stack Operations:
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display All
Select option: 6

Displaying Stacks...

30
20
10

Stack Operations:
1. Push
2. Pop
3. Top
4. isEmpty
```

Part 4:

```
5. isFull
6. Display All
Select option: 3

The top element is 30.

Stack Operations:
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display All
Select option: 2

Popping 30...

Stack Operations:
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display All
Select option: 6

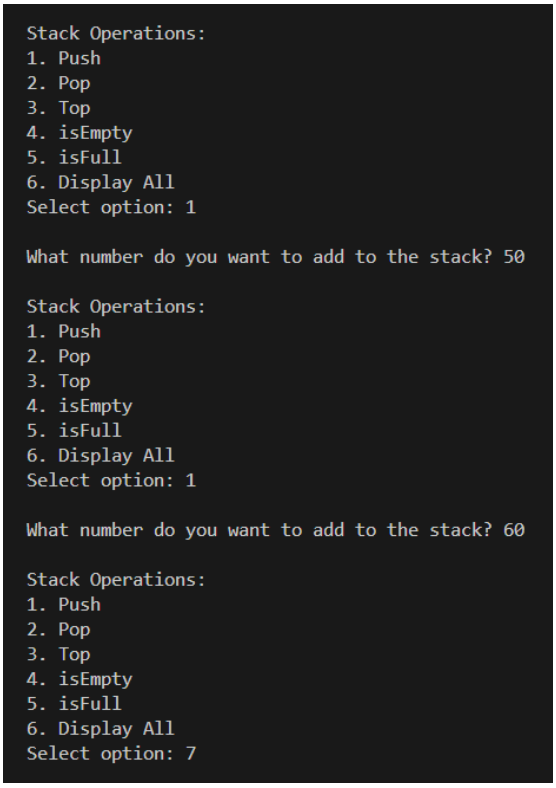
Displaying Stacks...

20
10

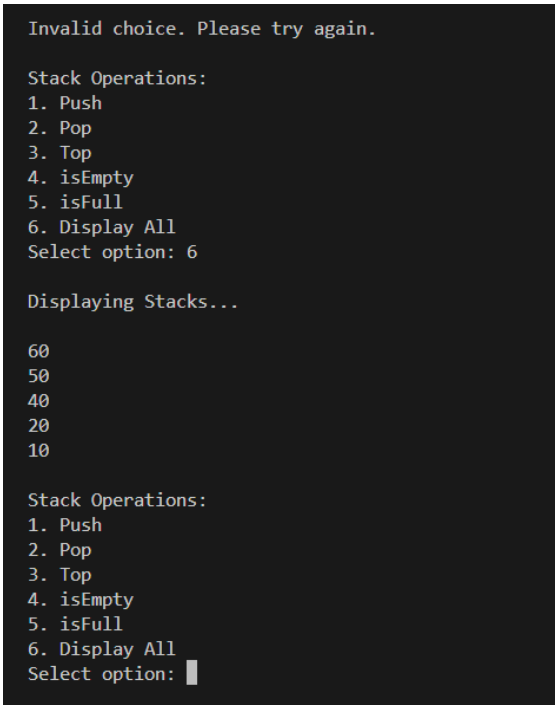
Stack Operations:
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display All
Select option: 1

What number do you want to add to the stack? 40
```

Part 5:



Part 6:



My Analysis:

The code shows how a basic stack was created using an array, where ***int stack[maxCap]*** is used to store the elements and ***top = -1*** shows that the stack is empty at the beginning. The important functions used are ***push()***, ***pop()***, ***Top()***, ***isEmpty()***, i***sFull()***, and ***displayAll()***, which handle the main operations of the stack. The ***push()*** function adds a new value using ***stack[++top] = newData***, but first ***isFull()*** checks if there is still space to avoid stack overflow. The ***pop()*** function removes the top element by using ***top--***, while ***isEmpty()*** checks first if there is an element that can be removed. The ***Top()*** function displays the current top value without removing it, which shows the value that will be removed first because the stack follows the ***LIFO (Last In, First Out)*** rule. The ***isEmpty()*** function checks if ***top == -1***, while ***isFull()*** checks if ***top >= elements - 1*** to prevent adding more than the selected maximum number of elements. The most important added part for the task is the ***displayAll()*** function, where ***for (int i = top; i >= 0; i--)*** is used to display all the elements starting from the top going down to the bottom. The ***switch(choice)*** inside the ***while(true)*** loop also allows different stack operations to be tested through the menu, making it easier to see how the stack changes after each operation.

B.2. Stacks using Linked Lists

Tasks:

- Modify the code given above to include a function that will display all elements in the stack.
- Provide a description of each operation provided.
- Include your output in the output section of your activity template.

My Code: (listStack.h)

ILOB2 > C listStack.h > ...

```
1   #ifndef LIST_STACK_H
2   #define LIST_STACK_H
3
4   #include <iostream>
5
6   // include node with data and next members
7   template <typename T>
8   class Node {
9       public:
10      T data;
11      Node<T>* next = nullptr;
12  };
13
14  // create a pointer/reference head that always points to the current top node
15  template <typename T>
16  Node<T>* head = nullptr;
17
18  // pushing operation:
19  template <typename T>
20  void push(T newData){
21      // create a new node
22      Node<T>* newNode = new Node<T>;
23
24      // assign the data to the new node
25      newNode->data = newData;
26
27      // point the new node to the current top node
28      newNode->next = head<T>;
29
30      // assign the new top to the new node
31      head<T> = newNode;
32  }
33
34  // isEmpty()
35  template <typename T>
```

```
36  bool isEmpty(){
37      // check if the head is pointing to nothing
38      return head<T> == nullptr;
39  }
40
41  // pop() operation:
42  template <typename T>
43  T pop(){
44      // if the stack is empty, display an error message and return an empty value
45      if(isEmpty<T>()){
46          std::cout << "Stack underflow." << std::endl;
47          return T();
48      }
49
50      // save the current top node to a temporary variable
51      Node<T>* temp = head<T>;
52
53      // store the data of the current top node
54      T tempVal = temp->data;
55
56      // move the head to the next node
57      head<T> = head<T>->next;
58
59      // free up the memory of the removed node
60      delete temp;
61
62      // return the data of the removed node
63      return tempVal;
64  }
65
66  // Top() operation:
67  template <typename T>
68  void Top(){
69      // check if the stack is empty
70      if(isEmpty<T>()){
71          std::cout << "Stack Underflow." << std::endl;
72          return;
```

```
73      }
74
75      // display the data of the current top node
76      std::cout << "\nCurrent Top: " << head<T>->data << std::endl;
77  }
78
79  // displayAll() operation:
80  template <typename T>
```

```
81  void displayAll(){
82
83      // check if the stack is empty
84      if(isEmpty<T>()){
85          std::cout << "\nList is empty." << std::endl;
86          return;
87      }
88
89      // create a temporary pointer starting from the current top
90      Node<T>* currNode = head<T>;
91
92      // display all elements without changing the actual head
93      std::cout << "\nDisplaying elements..." << std::endl;
94
95      while(currNode != nullptr){
96
97          // display the data of the current node
98          std::cout << currNode->data;
99
100         // display an arrow if another node exists
101         if(currNode->next != nullptr){
102             std::cout << " -> ";
103         }
104
105         // move to the next node
106         currNode = currNode->next;
107     }
108
109     std::cout << std::endl;
110 }
111
112 #endif
```

My Code: (main_ILOB2.cpp)

ILOB2 > main_ILOB2.cpp > main()

```cpp
#include <iostream>
#include "listStack.h"

int main(){

    std::cout << "\n===== Pushing 'I' 'E' 'E' 'E' ===== " << std::endl;

    // pushing
    push('I');
    push('E');
    push('E');
    push('E');

    // display top
    Top<char>();

    // displaying all elements
    displayAll<char>();

    // pop and print each element
    std::cout << "\n===== Popping 'E' 'E' 'E' 'I' =====" << std::endl;

    while(!isEmpty<char>()){
        std::cout << "\nPopping: " << pop<char>() << "..." << std::endl;
    }

    // test stack underflow
    std::cout << "\n---------------------------------" << std::endl;
    std::cout << pop<char>() << std::endl;

    return 0;
}
```

My Output:

```
===== Pushing 'I' 'E' 'E' 'E' =====

Current Top: E

Displaying elements...
E -> E -> E -> I

===== Popping 'E' 'E' 'E' 'I' =====

Popping: E...

Popping: E...

Popping: E...

Popping: I...

---------------------------------
Stack underflow.
```

My Analysis:

The code uses a linked list to create a stack, where each node contains ***data*** and a ***next pointer*** that connects it to the next node. The important part at the beginning is ***Node<T>***, which creates the structure of each node, while ***head<T> = nullptr*** is used to point to the current top of the stack. The ***push()*** function is important because it creates a new node using **new Node<T>**, stores the value using ***newNode->data = newData***, and connects it to the previous top through ***newNode->next = head<T>***, then ***head<T> = newNode*** makes the new node the top. The ***isEmpty()*** function checks if ***head<T> == nullptr***, which means there are no elements in the stack. The ***pop()*** function removes the current top node by first saving it in ***temp***, getting its value, moving ***head<T>*** to the next node, and using ***delete temp*** to free the removed node from memory. The ***Top()*** function

uses ***head<T>->data*** to display the current top without removing it.

The most important added part for the task is the ***displayAll()*** function, which uses ***Node<T>* currNode = head<T>*** and a ***while loop*** to go through every node and display the values using arrows like ***E -> E -> E -> I***. This also keeps the original ***head<T>*** unchanged because only ***currNode*** is moved while displaying. In ***main()***, the values ***'I', 'E', 'E'***, and ***'E'*** are pushed, showing the ***LIFO behavior*** because the last ***'E'*** becomes the top and is removed first. The ***while(!isEmpty<char>())*** loop then removes all elements in order, and the last ***pop<char>()*** tests the ***underflow condition*** after the stack becomes empty.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack:

The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:

a. Stack using Arrays
b. Stack using Linked Lists
c. (Optional) Stack using C++ STL

Problem Definition:

Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.

Steps:

1. Create a Stack.
2. While(end of input is not reached) {
    a) If the character read is not a symbol to be balanced, ignore it.
    b) If the character is an opening symbol, push it onto the stack.
    c) If it is a closing symbol:
        I. Report an error if the stack is empty.
        II. Otherwise, pop the stack.
    d) If the symbol popped is not the corresponding opening symbol, report an error.
3. At the end of input, if the stack is not empty: report an error.

Self-Checking:

For the following cases, complete the table using the code you created.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)			
((A+B)+(C-D)			
((A+B)+[C-D])			
((A+B]+[C-D]}`			

Tools Analysis:

- How do the different internal representations affect the implementation and usage of the stack?

My Code: (Stack_ARR_ILOC.h)

ILOC > Stack_ARR_ILOC.h > push(char)

```
1   #ifndef STACK_ARR_ILOC_H
2   #define STACK_ARR_ILOC_H
3
4   #include <iostream>
5   #include <string>
6
7   // Maximum number of symbols that can be stored in the stack
8   const int MAX_SIZE = 100;
9
10  // Array used to store the stack elements
11  char stack[MAX_SIZE];
12
13  // Index that points to the top of the stack
14  int top = -1;
15
16  // Push an opening symbol onto the stack
17  void push(char newData) {
18
19      // Check if the stack is full
20      if (top == MAX_SIZE - 1) {
21          std::cout << "Stack overflow." << std::endl;
```

```
22          return;
23      }
24
25      // Move the top index to the next position
26      top++;
27
28      // Store the symbol in the stack
29      stack[top] = newData;
30  }
31
32  // Pop the top symbol from the stack
33  char pop() {
34
35      // Store the top symbol
36      char tempVal = stack[top];
37
38      // Move the top index down
39      top--;
40
41      // Return the symbol that was removed
42      return tempVal;
43  }
44
45  // Check if the stack is empty
46  bool isEmpty() {
47
48      // Return true if the top index is -1
49      return top == -1;
50  }
51
52  // Reset the stack before checking a new expression
53  void resetStack() {
54
55      // Reset the top index to -1
56      top = -1;
57  }
```

```
58
59  // Check if the expression contains balanced symbols
60  bool checkBalanced(std::string expr) {
61
62      // Reset the stack for every new expression
63      resetStack();
64
65      // Read the expression one character at a time
66      for (char c : expr) {
67
```

```
68          // Push an opening symbol onto the stack
69          if (c == '(' || c == '{' || c == '[') {
70              push(c);
71          }
72
73          // Check a closing symbol against the top of the stack
74          else if (c == ')' || c == '}' || c == ']') {
75
76              // Report an error if there is no opening symbol to match
77              if (isEmpty()) {
78                  std::cout << "Closing symbol has no matching opening symbol."
79                            << std::endl;
80                  return false;
81              }
82
83              // Remove the top opening symbol from the stack
84              char topSymbol = pop();
85
86              // Check if the opening and closing symbols match
87              if ((c == ')' && topSymbol != '(') ||
88                  (c == '}' && topSymbol != '{') ||
89                  (c == ']' && topSymbol != '[')) {
90
91                  // Report an error if the symbols do not match
92                  std::cout << "Symbols do not match." << std::endl;
93                  return false;
94              }
95          }
96
97          // Ignore all characters that are not symbols to be balanced
98      }
99
100     // Report an error if an opening symbol remains in the stack
101     if (!isEmpty()) {
```

```
102         std::cout << "Opening symbol has no matching closing symbol."
103                   << std::endl;
104         return false;
105     }
106
107     // Return true when all symbols are properly matched
108     return true;
109 }
110
111 #endif
```

My Code: (main_ARR_ILOC.cpp)

ILOC > main_ARR_ILOC.cpp > main()

```cpp
#include <iostream>
#include <string>
#include "Stack_ARR_ILOC.h"

int main() {

    int choice;
    std::string expr;

    do {

        std::cout << "\n================ BALANCED SYMBOLS MENU ================" << std::endl;
        std::cout << "-------------------------------------------------------" << std::endl;
        std::cout << "(1) Check Balanced Symbols" << std::endl;
        std::cout << "(2) Exit" << std::endl;
        std::cout << "Select an option: ";
        std::cin >> choice;
        std::cin.ignore();

        switch (choice) {

            // Check the expression for balanced symbols
            case 1: {
                std::cout << "\nEnter an expression: ";
                std::getline(std::cin, expr);

                // Display the result of the balanced-symbol check
                if (checkBalanced(expr)) {
                    std::cout << "Expression is Balanced!" << std::endl;
                }
                else {
                    std::cout << "Expression is not Balanced!" << std::endl;
                }

                break;
            }

            // Exit the program
            case 2:
                std::cout << "\nExiting balanced symbols checker..." << std::endl;
                break;

            // Handle an invalid menu option
            default:
                std::cout << "\nInvalid option. Please try again." << std::endl;
        }

    }
    while (choice != 2);

    return 0;
}
```

My Code: (Stack_LL_ILOC.h)

ILOC > C Stack_LL_ILOC.h > ...

```
1   #ifndef STACK_LL_ILOC_H
2   #define STACK_LL_ILOC_H
3
4   #include <iostream>
5   #include <string>
6
7   // Node for the linked list stack
8   struct Node {
9       char data;
10      Node* next = nullptr;
11  };
12
13  // Global pointer that points to the top of the stack
14  Node* topNode = nullptr;
15
16  // Push an opening symbol onto the stack
17  void push(char newData) {
18
19      // Create a new node
20      Node* newNode = new Node();
21
22      // Store the symbol in the new node
23      newNode->data = newData;
24
25      // Link the new node to the current top node
26      newNode->next = topNode;
27
28      // Make the new node the top of the stack
29      topNode = newNode;
30  }
31
32  // Pop the top symbol from the stack
33  char pop() {
34
35      // Store the current top node in a temporary pointer
36      Node* temp = topNode;
37
38      // Store the data of the top node
39      char tempVal = temp->data;
```

```
40
41      // Move the top pointer to the next node
42      topNode = topNode->next;
43
44      // Delete the old top node to free memory
45      delete temp;
46
47      // Return the symbol that was removed
48      return tempVal;
49  }
50
51  // Check if the stack is empty
52  bool isEmpty() {
53
54      // Return true if there is no node in the stack
55      return topNode == nullptr;
56  }
57
58  // Reset the stack before checking a new expression
59  void resetStack() {
60
61      // Remove all remaining nodes from the stack
62      while (topNode != nullptr) {
63
64          // Store the current top node temporarily
65          Node* temp = topNode;
66
67          // Move the top pointer to the next node
68          topNode = topNode->next;
69
70          // Free the removed node from memory
71          delete temp;
72      }
73  }
74
```

```
75  // Check if the expression contains balanced symbols
76  bool checkBalanced(std::string expr) {
77
78      // Reset the stack for every new expression
79      resetStack();
80
81      // Read the expression one character at a time
82      for (char c : expr) {
83
84          // Push an opening symbol onto the stack
```

```
85          if (c == '(' || c == '{' || c == '[') {
86              push(c);
87          }
88
89          // Check a closing symbol against the top of the stack
90          else if (c == ')' || c == '}' || c == ']') {
91
92              // Report an error if there is no opening symbol to match
93              if (isEmpty()) {
94                  std::cout << "Closing symbol has no matching opening symbol."
95                            << std::endl;
96                  return false;
97              }
98
99              // Remove the top opening symbol from the stack
100             char top = pop();
101
102             // Check if the opening and closing symbols match
103             if ((c == ')' && top != '(') ||
104                 (c == '}' && top != '{') ||
105                 (c == ']' && top != '[')) {
106
107                 // Report an error if the symbols do not match
108                 std::cout << "Symbols do not match." << std::endl;
109                 return false;
110             }
111         }
112
113         // Ignore all characters that are not symbols to be balanced
114     }
115
116     // Report an error if an opening symbol remains in the stack
117     if (!isEmpty()) {
118         std::cout << "Opening symbol has no matching closing symbol."
119                   << std::endl;
```

```
120         return false;
121     }
122
123     // Return true when all symbols are properly matched
124     return true;
125 }
126
127 #endif
```

My Code: (main_LL_ILOC.cpp)

ILOC > main_LL_ILOC.cpp > main()

```cpp
#include <iostream>
#include <string>
#include "Stack_LL_ILOC.h"

int main() {

    int choice;
    std::string expr;

    do {

        std::cout << "\n================ BALANCED SYMBOLS MENU ================" << std::endl;
        std::cout << "-------------------------------------------------------" << std::endl;
        std::cout << "(1) Check Balanced Symbols" << std::endl;
        std::cout << "(2) Exit" << std::endl;
        std::cout << "Select an option: ";
        std::cin >> choice;
        std::cin.ignore();

        switch (choice) {

            // Check the expression for balanced symbols
            case 1: {
                std::cout << "\nEnter an expression: ";
                std::getline(std::cin, expr);

                // Display the result of the balanced-symbol check
                if (checkBalanced(expr)) {
                    std::cout << "Expression is Balanced!" << std::endl;
                }
                else {
                    std::cout << "Expression is not Balanced!" << std::endl;
                }

                break;
            }

            // Exit the program
            case 2:
                std::cout << "\nExiting balanced symbols checker..." << std::endl;
                break;

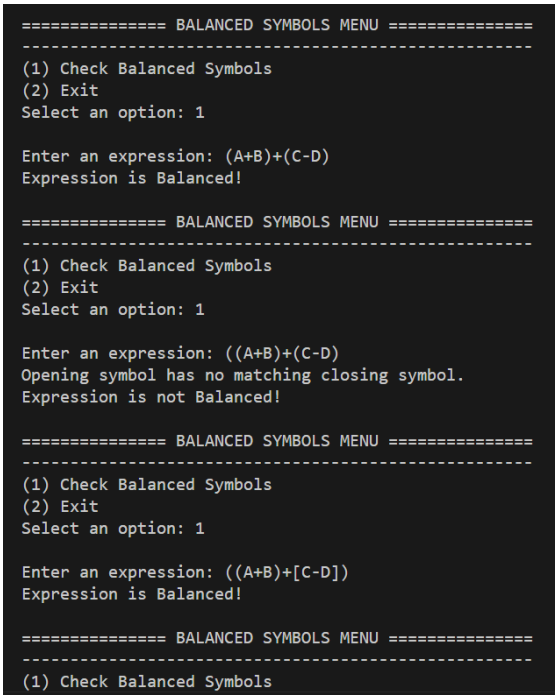
            // Handle an invalid menu option
            default:
                std::cout << "\nInvalid option. Please try again." << std::endl;
        }

    } \
    while (choice != 2);

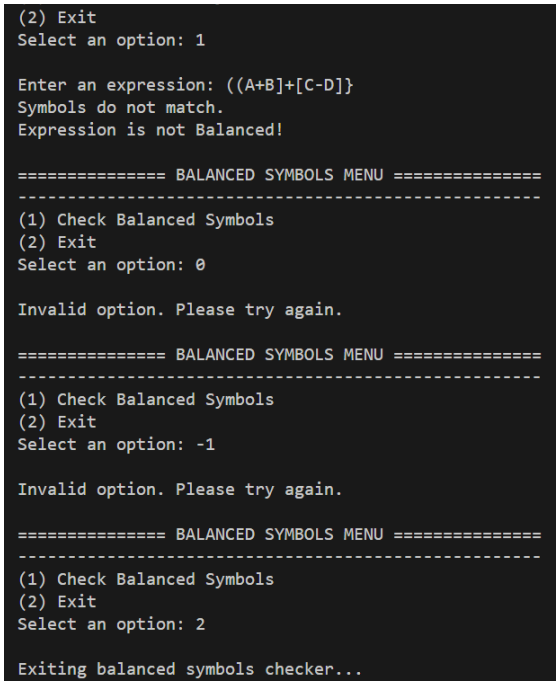
    return 0;
}
```

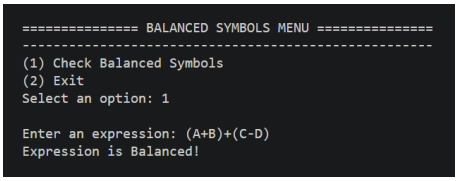
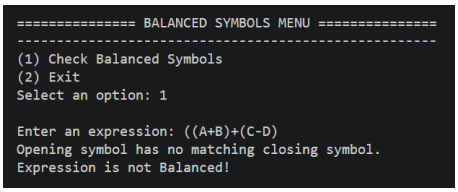
My Output: (Array Stack = Linked List Stack)

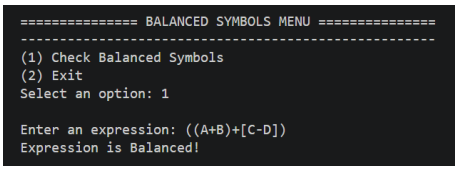
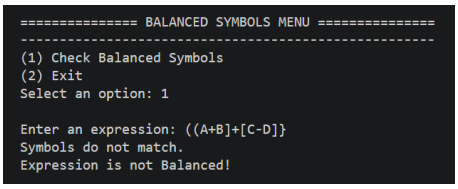
Part 1



Part 2



Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	**Y**		All opening symbols have the correct closing symbols. The **(** is matched with **)**, and the expression has no extra symbols left in the stack.
((A+B)+(C-D)	**N**		There is one extra **(** that remains in the stack after checking the whole expression, so it does not have a matching **)**.

((A+B)+[C-D])	**Y**		The parentheses and brackets are properly matched. The **(** matches **)**, while **[** matches **]**, so the stack becomes empty at the end.
((A+B]+[C-D]}	**N**		The closing **]** is encountered while **(** is the symbol on top of the stack, so the symbols do not match. The expression is therefore not balanced.

My Analysis:

The code uses two different stack implementations, one using an array and another using a linked list, to check if symbols in an expression are balanced. In the array version, ***char stack[MAX_SIZE]*** stores the symbols and ***top = -1*** keeps track of the current top position, while in the linked-list version, ***Node*** contains ***data*** and ***next***, and ***topNode*** points to the current top node. The important ***push()*** function adds opening symbols such as ***(***, ***{***, and ***[*** to the stack, while the ***pop()*** function removes the latest opening symbol when a closing symbol is found. The ***isEmpty()*** function is also important because it checks if there is an opening symbol available before calling ***pop()***, which helps detect when a closing symbol has no matching opening symbol. The ***resetStack()*** function is also important because it clears the previous stack before checking a new expression.

In the array version, ***top = -1*** makes the stack empty again, while in the linked-list version, the ***while*** loop together with ***delete temp*** removes the old nodes and frees their memory. The main checking process is handled by ***checkBalanced(std::string expr)***, where the expression is read one character at a time using for ***(char c : expr)***, while normal characters like letters and operators are ignored. When an opening symbol is found, ***push(c)*** is used, and when a closing symbol is found, the top symbol is removed using ***pop()*** and compared with the closing symbol through conditions such as ***(c == ')' && topSymbol != '(')***, which detects mismatched symbols. At the end, ***if (!isEmpty())*** checks if there are still opening symbols left in the stack, which means some symbols do not have a matching closing symbol.

For the first expression, ***(A+B)+(C-D)***, the result is ***valid (Y)*** because all opening and closing parentheses match correctly, so the stack becomes empty and the output is "***Expression is Balanced!***". For ***((A+B)+(C-D)***, the result is ***not valid (N)*** because one opening ***(*** remains in the stack without a matching ***)***, resulting in the "***Opening symbol having no matching closing symbol***." and "***Expression is not Balanced!***". For ***((A+B)+[C-D])***, the result is ***valid (Y)*** because the parentheses and brackets are matched correctly, so the output is "***Expression is Balanced!***". Lastly, ***((A+B]+[C-D]}*** is ***not valid (N)*** because a closing ***]*** is found while ***(*** is at the top of the stack, causing "***Symbols do not match***." and "***Expression is not Balanced!***". The ***main()*** function provides a simple menu where an expression can be entered and checked, making the program easier to test.

Conclusion:

The activity helped show how stacks work and how they can be applied to different problems using arrays, linked lists, and the C++ STL. The main lesson learned was the LIFO (Last In, First Out) principle, where the latest element added is the first one removed. The procedure also helped in understanding the important stack operations such as push(), pop(), top(), empty(), size(), emplace(), swap(), isEmpty(), and isFull(). In the array implementation, top was used to keep track of the current position, while the linked-list implementation used nodes and pointers to connect the elements. The procedure also showed that different internal representations can still follow the same stack behavior even if the way of storing the elements is different. For the supplementary activity, the balanced-symbol problem helped show a more practical use of stacks. The checkBalanced() function reads each character, pushes opening symbols like (, {, and [, then pops and compares them when closing symbols are found. The given expressions also helped in understanding how missing or mismatched symbols are detected.

One difficulty faced was keeping track of the top element after every push() and pop(), especially when several different symbols were used in one expression. Another difficulty was understanding the resetStack() function, particularly in the linked-list version where the old nodes need to be removed using delete temp so they do not stay in memory. The array version was easier to follow because it only needed top = -1 to reset the stack, but it also has a fixed MAX_SIZE limit. The activity was done fairly well because the required stack implementations were created and the test expressions were checked with the expected results. However, there are still areas for improvement, especially in understanding pointers, dynamic memory, and how the stack changes internally after each operation. More practice with linked lists and tracing the values of top and topNode step by step would help make the process more familiar.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **formatting, explanation of concepts, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to explain how the code works by giving a detailed explanation on**

the different parts of the code. Moreover, the AI was used to assist in improving the grammar of the code analysis, allowing the explanation to be written more clearly and fluently in English.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.